

第 5 集 · 约 18 分钟

分块传输与抓包测试

1.5 GB 的视频怎么传？“加密了”又怎么证明？

配套阅读：书的第 5 章



这一集你会学到

1. 大文件切成 1 MiB 一块：按偏移写，可以乱序、重发、续传
2. `io::Error` 的 `kind()`：把系统错误翻译成简短的错误码
3. `move` 闭包：把数据的所有权交给另一个线程
4. 抓包测试：用代码扮演同一个 WiFi 下的偷看者
5. 一个真实的坑：Future 是惰性的，看起来并发其实是串行

WiFi 断了一下

1.5 GB 的视频传到一半，WiFi 断了一下。要从头再传吗？

v1 要：一个请求传整个文件。v2 不用：文件切成 1 MiB 一块，每块单独加密、单独请求，断了只补没收到的那几块。

整个文件 vs 一块一块

Python v1 · storage.py

```

1 def save_stream(self, name, stream, length):
2     safe = sanitize_filename(name)
3     tmp = os.path.join(self.root, TEMP_PREFIX + secrets.token_hex(16))
4     try:
5         with open(tmp, "wb") as f:           # with: 块
6             remaining = length
7             while remaining > 0:
8                 chunk = stream.read(min(CHUNK_SIZE, remaining))
9                 if not chunk:
10                     raise IncompleteUpload("还差 %d 字节" % remaining)
11                 f.write(chunk)
12                 remaining -= len(chunk)
13         with self._lock:
14             final = self._free_path(safe)
15             os.rename(tmp, final)
16         return os.path.basename(final)
17     except BaseException:
18         _remove_quietly(tmp)           # 出任何错都
19         raise

```

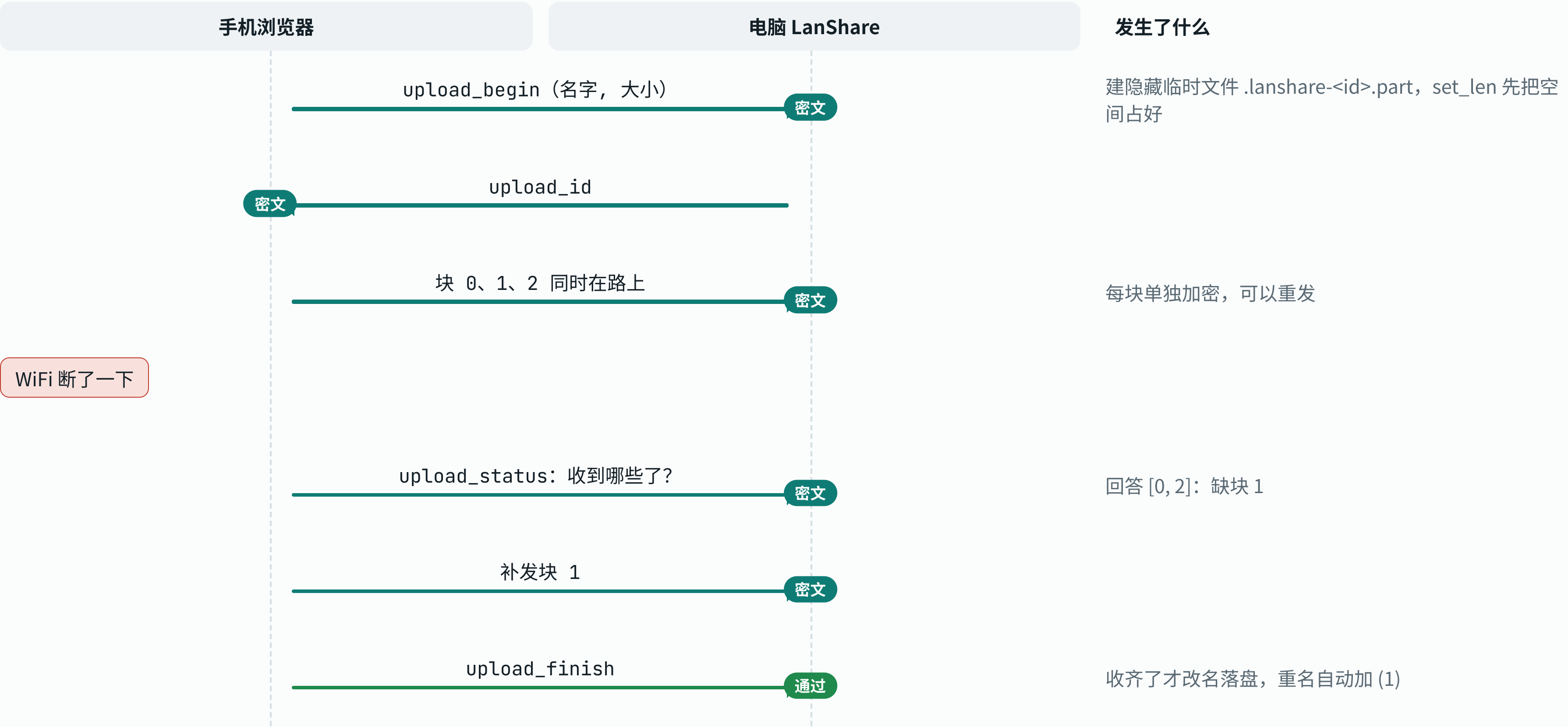
Rust v2 · server/transfer.rs

```

1 let written = tokio::task::spawn_blocking(move || -> io::Write::write_all(&data, &file)).await;
2     let mut file = OpenOptions::new().write(true).open(&path).await?;
3     file.seek(SeekFrom::Start(index as u64 * CHUNK as u64)).await?;
4     file.write_all(&data)
5 }
6 .await;

```

一次分块上传



动手：分块、断网、续传

手机上的文件（每块 1 MiB，共 12 块）

0

1

2

3

4

5

6

7

8

9

10

11

路上（最多 3 块同时在途，每块单独加密）

电脑上的临时文件 .lanshare-...part（按块号写到对应位置）

.

.

.

.

.

.

.

.

.

.

.

.

开始上传

断网

恢复网络，续传

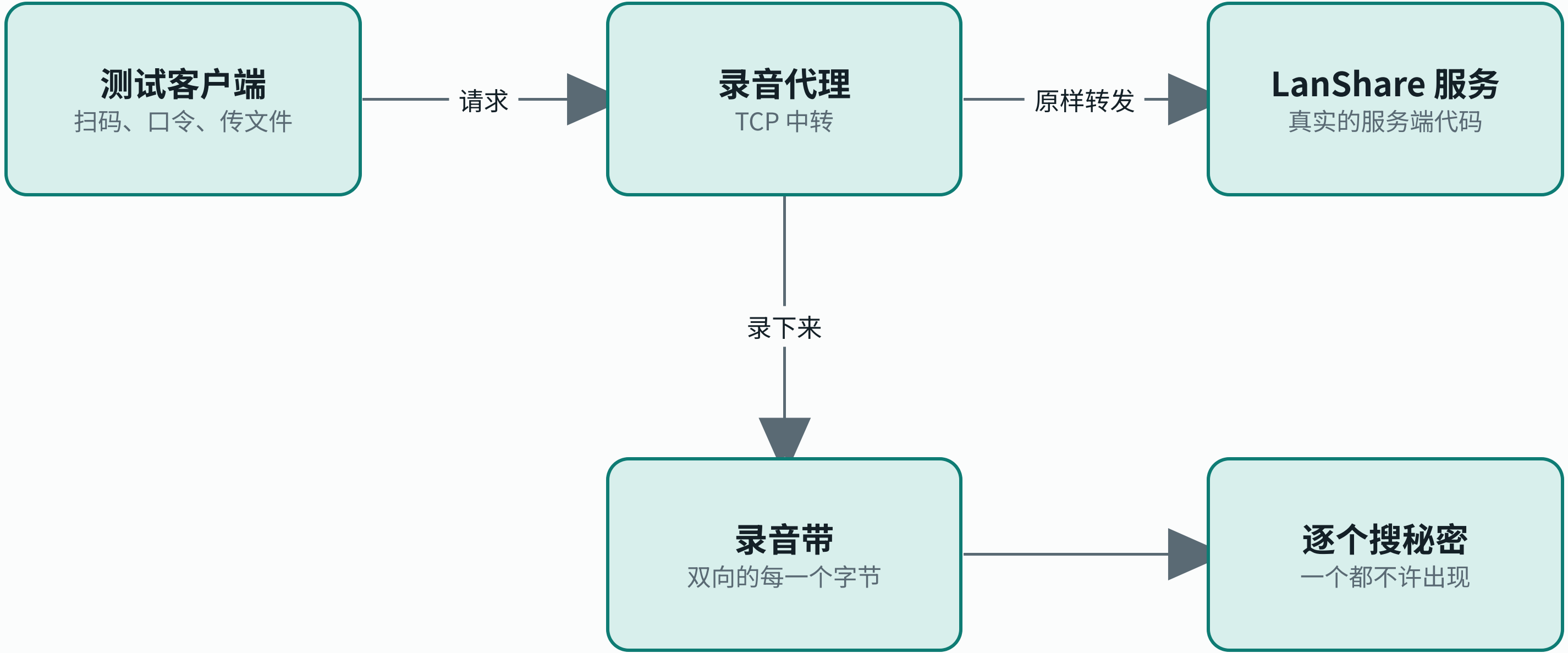
重来

move: 交出所有权

```
crates/lanshare/src/server/transfer.rs
```

```
1 if data.len() as u64 != expected_len(size, index) {
2     return reply_error("bad_length");
3 }
4 let written = tokio::task::spawn_blocking(move || -> io::Result<()> {
5     let mut file = OpenOptions::new().write(true).open(&temp)?;
6     file.seek(SeekFrom::Start(index as u64 * CHUNK as u64))?;
7     file.write_all(&data)
8 })
9 .await;
10 match written {
11     Ok(Ok(())) => {
12         let mut uploads = st.uploads.lock().expect("锁不会中毒");
13         if let Some(upload) = uploads.0.get_mut(upload_id) {
14             upload.received[index] = true;
15             upload.last_active = Instant::now();
16         }
17         reply(json!({ "ok": true }))
18     }
19     Ok(Err(e)) => reply_error(io_error_code(&e)),
20     Err(_) => reply_error("io_error"),
```

用代码扮演偷看者



录音代理的核心

crates/lanshare/tests/wire.rs

```
1  async fn pump(  
2      mut from: tokio::net::tcp::OwnedReadHalf,  
3      mut to: tokio::net::tcp::OwnedWriteHalf,  
4      tape: Arc<Mutex<Vec<u8>>>,  
5  ) {  
6      let mut buf = vec![0u8; 64 * 1024];  
7      loop {  
8          match from.read(&mut buf).await {  
9              Ok(0) | Err(_) => break,  
10             Ok(n) => {  
11                 tape.lock().unwrap().extend_from_slice(&buf[..n]);  
12                 if to.write_all(&buf[..n]).await.is_err() {  
13                     break;  
14                 }  
15             }  
16         }  
17     }  
18     let _ = to.shutdown().await;  
19 }
```


在录音里搜秘密

搜索的内容	结果
二维码密钥 K（原始字节 / hex / base64url）	找不到
口令 PIN	找不到
会话秘密、方向密钥	找不到
文件名、文字内容	找不到
文件开头、中间、结尾各 64 字节	找不到
二维码 SVG 的 <code><svg</code> 标签	找不到
反向自检：录音总长度	大于文件大小的 2 倍
反向自检：请求行 <code>/api/hello</code> 、会话 id	能找到

跑一下试试

Windows PowerShell

```
PS D:\lanshare> cargo test -p lanshare --test transfer resume
running 1 test
test resume_after_interruption ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 10 filtered out; finished in 0.07s

PS D:\lanshare> cargo test -p lanshare --test wire -- --nocapture
running 1 test
test eavesdropper_sees_no_secrets ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 1.22s

PS D:\lanshare> cargo test -p lanshare --test wire -- --nocapture
running 1 test

thread 'eavesdropper_sees_no_secrets' (50440) panicked at crates\lanshare\tests\common\mod.rs:131:77:
响应必须能用 s2c 解开: Decrypt
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
test eavesdropper_sees_no_secrets ... FAILED

failures:
```

踩坑：并发其实是串行

示意 · 先收集 future，再逐个 await

```
1 let futures: Vec<_> = urls.iter().map(|u| client.get(u).send().await).collect();
2 for f in futures { f.await; }
```

现在 · tests/transfer.rs

```
1 let tasks: Vec<_> = sealed
2     .into_iter()
3     .rev()
4     .map(|(path, ctr, body)| {
5         let (http, base, sid) = (c.http.clone(), c.base.clone(), ctr);
6         tokio::spawn(async move {
7             http.post(format!("{base}{path}"))
8                 // ...
9                 .send()
10                .await
11                .unwrap()
12                .status()
13                .as_u16()
14            })
15    })
16    .collect();
17 let mut statuses = Vec::new();
18 for task in tasks {
19     statuses.push(task.await.unwrap());
20 }
```

选择题

下面的代码会同时发出 4 个请求吗？

```
1 let futures: Vec<_> = urls.iter().map(|u| client.get(u).send()).collect();  
2 for f in futures { f.await; }
```

A 会，4 个请求同时发出

B 不会，实际是一个接一个地发

C 会，只是完成的先后顺序随机

D 编译不过

选择题

下面的代码会同时发出 4 个请求吗？

```
1 let futures: Vec<_> = urls.iter().map(|u| client.get(u).send()).collect();  
2 for f in futures { f.await; }
```

A 会，4 个请求同时发出

send() 只是创建了 Future，要等被 .await（也就是被 poll）才开始执行。逐个 await 就是逐个发。

B 不会，实际是一个接一个地发

对。Rust 的 Future 是惰性的。想要并发，可以用 tokio::spawn，或者会同时 poll 所有 future 的 join_all。

C 会，只是完成的先后顺序随机

这是 JavaScript Promise 的直觉，一创建就开始跑。Rust 的 Future 不是这样。

D 编译不过

能编译。坑就坑在它能跑、结果也对，只是悄悄变成了串行。

选择题

把 `spawn_blocking(move || ...)` 里的 `move` 去掉，会怎样？

A 能编译，Rust 会自动把 data 复制一份给闭包

B 能编译，但两个线程可能同时改 data

C 编译不过，闭包不能只借用外面的变量

D 能编译，到运行时才报错

选择题

把 `spawn_blocking(move || ...)` 里的 `move` 去掉，会怎样？

A 能编译，Rust 会自动把 data 复制一份给闭包

Rust 不会悄悄复制 1 MiB 的数据。想复制得自己写 `clone()`。

B 能编译，但两个线程可能同时改 data

这正是 Rust 在编译期就拦住的事，根本跑不起来。

C 编译不过，闭包不能只借用外面的变量

对。闭包要在另一个线程上跑，可能比当前函数活得还久，所以必须把路径和数据的所有权移进去，移进去之后外面就不能再用。

D 能编译，到运行时才报错

问题在编译期就会被发现，不用等到运行。

选择题

别的设备拿着你的 `upload_id` 来写数据，服务端为什么回答 `not_found`，而不是“没有权限”？

```
1 let Some(upload) = uploads.0.get_mut(upload_id).filter(|u| u.owner == cid) else {  
2     return reply_error("not_found");  
3 };
```

A 这是个 bug，本来应该返回“没有权限”

B 因为服务端的表里确实查不到这个 id

C 为了少定义一个错误码

D 说“没有权限”，等于承认这个 id 存在

选择题

别的设备拿着你的 `upload_id` 来写数据，服务端为什么回答 `not_found`，而不是“没有权限”？

```
1 let Some(upload) = uploads.0.get_mut(upload_id).filter(|u| u.owner == cid) else {
2     return reply_error("not_found");
3 };
```

A 这是个 bug，本来应该返回“没有权限”

是故意这么写的。测试 `another_device_cannot_touch_my_upload` 专门检查别的设备拿到的是 `not_found`。

B 因为服务端的表里确实查不到这个 id

表里有这个 id，只是它的 `owner` 是另一个会话，`filter` 把它筛掉了。

C 为了少定义一个错误码

不是为了省事，是为了不透露任何信息。

D 说“没有权限”，等于承认这个 id 存在

对。那等于告诉对方“猜对了”。统一说不存在，什么都不泄露。`Option::filter` 的意思是：存在、并且满足条件才保留，否则一律当成没有。

想一想

抓包测试全绿了，能不能说 LanShare “绝对安全”了？还缺什么？

想一想

抓包测试全绿了，能不能说 LanShare “绝对安全”了？还缺什么？

不能。抓包测试证明的是“网络上看不到秘密”这一件事。它不管别人能不能把服务拖慢、能不能把内存或磁盘撑满。而且这些测试都是写代码的人自己写的：想到的攻击防了，没想到的，测试里也不会有。所以第 9 集请了一个“外人”来专门攻击它，真的找出了问题，比如没传完的上传永不过期。

这一集我们学了

- 🚫 大文件切成 1 MiB 一块：按偏移写，可以乱序、重发、续传
- 🚫 `set_len` 先占空间；`io::Error` 按 `kind()` 翻译成错误码
- 🚫 `move` 闭包把所有权交给另一个线程，编译期保证没人同时碰它
- 🚫 抓包测试：录下每一个字节，逐个搜秘密，再用反向自检证明测试没空跑
- 🚫 `Future` 是惰性的：要真并发，用 `tokio::spawn`

下一集

下一集：跟着一条文字消息，从手机走到电脑，再走回来